

Mini-Projet Cloud

Application déployée :

Réalisé par	Rayen Othmani
Email	rayen.othmani@atlas-labs.org
Formation	Licence / Master — Informatique
Encadrant	Enseignant Cloud & DevOps — ITBS
Date	Mai 2026

Table des Matières

Introduction	3
1. Contexte du projet	4
1.1 Contexte & problématique	4
1.2 Objectifs du projet	4
1.3 Présentation de l'application déployée ..	5
2. Architecture globale du système	6
2.1 Vue d'ensemble — pipeline complet	6
2.2 Architecture Kubernetes	7
3. Conteneurisation avec Docker	8
3.1 Création des Dockerfiles	8
3.2 Build des images	10
3.3 Exécution locale	10
4. Orchestration avec Kubernetes	11
4.1 Création des fichiers YAML	11
4.2 Déploiement sur cluster local (Minikube) ..	13
4.3 Vérification	13
4.4 Accès à l'application	14
5. Déploiement sur GCP	15
5.1 Configuration initiale GCP	15
5.2 Registre d'images — Docker Hub	15
5.3 Création du cluster GKE Autopilot	16
5.4 Connexion & déploiement	16
5.5 Exposition de l'application	17
6. Résultats & validation	18
6.1 Preuves de déploiement	18
6.2 Application accessible	18
6.3 Difficultés rencontrées & solutions apportées ..	19
Conclusion & Perspectives	20

Introduction

Le domaine du déploiement applicatif a connu une transformation radicale ces dernières années. Les approches traditionnelles — reposant sur des serveurs physiques ou des machines virtuelles configurées manuellement — montrent leurs limites face aux exigences modernes de scalabilité, de portabilité et de résilience. Dans ce contexte, la conteneurisation avec **Docker** et l'orchestration avec **Kubernetes** sont devenus des standards incontournables de l'industrie.

Ce rapport présente le travail réalisé dans le cadre du mini-projet de Cloud Computing : le déploiement complet d'une application web full-stack — **User Role Management System** — depuis la conteneurisation jusqu'à la mise en production sur **Google Cloud Platform (GCP)** via **Google Kubernetes Engine (GKE)**.

L'application déployée est un système de gestion des utilisateurs et des rôles, développé avec Spring Boot 3.2 (Java 21) pour le backend, Angular 17 pour le frontend, et PostgreSQL 16 pour la persistance des données. Le pipeline DevOps mis en place intègre GitHub Actions pour l'intégration continue, Docker Hub pour le registre d'images, et ArgoCD pour le déploiement GitOps.

Ce rapport suit le plan fourni par l'enseignant et documente chaque étape du déploiement, accompagnée des commandes et configurations utilisées.

1. Contexte du Projet

1.1 Contexte & Problématique

Limites du déploiement traditionnel

Dans un déploiement traditionnel, chaque environnement (développement, test, production) doit être configuré manuellement. Cela entraîne plusieurs problèmes récurrents :

- **Dépendances non portables** : "ça marche sur ma machine" — le code fonctionne en développement mais échoue en production à cause de versions différentes de Java, Node.js ou des bibliothèques système.
- **Scalabilité difficile** : pour absorber un pic de trafic, il faut provisionner manuellement de nouveaux serveurs, ce qui est lent, coûteux et source d'erreurs.
- **Déploiements fragiles** : les déploiements manuels sont non reproductibles. Une mise à jour peut casser la production sans rollback facile.
- **Isolation insuffisante** : plusieurs applications partageant le même serveur peuvent créer des conflits de dépendances ou de ressources.

Motivation pour l'approche conteneurisée et cloud

La conteneurisation avec Docker résout le problème de portabilité en encapsulant l'application et toutes ses dépendances dans une image immuable. Kubernetes automatise le déploiement, la scalabilité et la gestion des conteneurs. GCP fournit l'infrastructure cloud managée (GKE) qui élimine la gestion des serveurs physiques.

1.2 Objectifs du Projet

- Conteneuriser les trois composants de l'application (backend, frontend, base de données) avec des Dockerfiles multi-étapes optimisés.
- Orchestrer le déploiement avec Kubernetes en définissant Deployments, Services, ConfigMaps, Secrets et PersistentVolumeClaims.
- Déployer l'application sur un cluster GKE Autopilot dans la région europe-west1 (Belgique).
- Exposer l'application publiquement via un Service LoadBalancer de GCP.
- Mettre en place un pipeline CI/CD avec GitHub Actions et un déploiement GitOps avec ArgoCD.
- Documenter les difficultés rencontrées et les solutions apportées.

1.3 Présentation de l'Application Déployée

Type d'application

L'application est un système web full-stack de type **API REST + SPA (Single Page Application)**. Il s'agit d'un système de gestion des utilisateurs et des rôles avec authentification JWT et contrôle d'accès basé sur les rôles (RBAC — Role-Based Access Control).

Stack technologique

Couche	Technologie	Version
Frontend	Angular	17.3.0
Serveur frontend	NGINX	Alpine
Backend	Spring Boot / Java	3.2.0 / Java 21
Authentification	JWT (jjwt)	0.12.3
Base de données	PostgreSQL	16-Alpine
ORM	Spring Data JPA / Hibernate	—
Monitoring	Micrometer + Prometheus	—
Conteneurisation	Docker / Docker Compose	—
Orchestration	Kubernetes (GKE Autopilot)	—
CI/CD	GitHub Actions	—
GitOps	ArgoCD	—

Fonctionnalités principales

- **Authentification JWT** : inscription, connexion, génération et validation de tokens JWT.
- **Gestion des utilisateurs** : CRUD complet (création, lecture, mise à jour, suppression).
- **Gestion des rôles et permissions** : attribution de rôles aux utilisateurs, définition de permissions fines par rôle.
- **Historique d'audit** : journalisation des actions via Spring AOP.
- **Healthcheck** : endpoint /actuator/health exposé pour Kubernetes.
- **Métriques Prometheus** : endpoint /actuator/prometheus pour le monitoring.

2. Architecture Globale du Système

2.1 Vue d'Ensemble — Pipeline Complet

Le pipeline de déploiement suit le chemin suivant, du code source jusqu'à l'utilisateur final :



Figure 1 — Pipeline CI/CD complet : Code source → Build → Docker Hub → ArgoCD → GKE → Utilisateur

Description de chaque étape du pipeline

Étape	Description
1. Code source (GitHub)	Le code est versionné sur GitHub. Deux branches principales : main (production) et dev (développement)
2. GitHub Actions CI	À chaque push, le pipeline déclenche les tests unitaires, l'analyse de qualité (Checkstyle, ESLint), le scan de sécurité (OWASP)
3. Docker Hub	Registre central pour les images : raybob2001/user-role-management-backend:latest et raybob2001/user-role-management-frontend:latest
4. ArgoCD (GitOps)	ArgoCD surveille le dépôt GitHub (dossier k8s/). À chaque modification des manifestes YAML, il synchronise le cluster Kubernetes
5. GKE Autopilot	Cluster Kubernetes managé par Google dans la région europe-west1. GKE gère automatiquement le scaling et la maintenance des nœuds
6. Utilisateur	L'application est accessible via l'IP publique du Service LoadBalancer GCP sur le port 80.

2.2 Architecture Kubernetes



Figure 2 — Architecture Kubernetes dans le namespace user-role-management

Composants Kubernetes utilisés

Ressource	Nom	Type/Description
-----------	-----	------------------

Namespace	user-role-management	Isolation logique de tous les ressources
Deployment	frontend	2 réplicas — Angular servi par NGINX
Deployment	backend	2 réplicas — Spring Boot avec healthcheck
Deployment	postgres	1 réplica — PostgreSQL avec PVC
Service	frontend	LoadBalancer — IP publique GCP, port 80
Service	backend	ClusterIP — interne au cluster, port 8080
Service	postgres	ClusterIP — interne, port 5432
ConfigMap	app-config	DB_HOST, DB_PORT, DB_NAME
ConfigMap	frontend-nginx-config	Configuration NGINX avec proxy /api/
Secret	app-secret	DB_PASSWORD, JWT_SECRET (Base64)
PVC	postgres-pvc	Stockage persistant 1Gi pour PostgreSQL

Configuration NGINX (nginx.conf)

NGINX joue un double rôle : serveur de fichiers statiques Angular ET reverse proxy vers le backend.

3.2 Build des Images

Les images sont buildées et poussées vers Docker Hub via GitHub Actions CI/CD. Voici les commandes équivalentes pour un build manuel :

```
# Build de l'image backend
$ docker build -t raybob2001/user-role-management-backend:latest ./backend
[+] Building 145.2s (14/14) FINISHED
=> [internal] load build definition from Dockerfile
=> [build 1/4] FROM maven:3.9-eclipse-temurin-21
=> [build 2/4] COPY pom.xml .
=> [build 3/4] RUN mvn dependency:go-offline -B
=> [build 4/4] RUN mvn package -DskipTests -B
=> [runtime 1/3] FROM eclipse-temurin:21-jre-alpine
=> [runtime 2/3] COPY --from=build /app/target/*.jar app.jar
=> exporting to image
Successfully built a3f2b1c9d5e8
Successfully tagged raybob2001/user-role-management-backend:latest

# Build de l'image frontend
$ docker build -t raybob2001/user-role-management-frontend:latest ./frontend
[+] Building 89.3s (12/12) FINISHED
=> [build 1/4] FROM node:20-alpine
=> [build 2/4] RUN npm ci --ignore-scripts
=> [build 3/4] COPY . .
=> [build 4/4] RUN npm run build -- --configuration production
=> [runtime 1/2] FROM nginx:alpine
=> [runtime 2/2] COPY --from=build /app/dist/frontend/browser /usr/share/nginx/html
Successfully built b7e4d2a1f6c9
Successfully tagged raybob2001/user-role-management-frontend:latest

# Push vers Docker Hub
$ docker push raybob2001/user-role-management-backend:latest
$ docker push raybob2001/user-role-management-frontend:latest
```

3.3 Exécution Locale avec Docker Compose

Pour tester l'application localement avant le déploiement Kubernetes, un fichier **docker-compose.yml** orchestre les trois services (postgres, backend, frontend) :

```
# Démarrage de tous les services
$ docker-compose up -d
[+] Running 3/3
✓ Container user-role-management-postgres-1   Healthy
✓ Container user-role-management-backend-1    Started
✓ Container user-role-management-frontend-1   Started

# Vérification des conteneurs
$ docker-compose ps
NAME                                IMAGE                                STATUS      PORTS
postgres-1                         postgres:16-alpine                 Up (healthy) 5432/tcp
backend-1                          user-role-management-backend:latest Up           0.0.0.0:8080->8080/
frontend-1                          user-role-management-frontend:latest Up           0.0.0.0:3001->80/tc

# Test de l'API backend
$ curl http://localhost:8080/actuator/health
```

```
{"status": "UP", "components": {"db": {"status": "UP"}, "ping": {"status": "UP"}}}
```

```
# L'interface Angular est accessible sur : http://localhost:3001
```

4. Orchestration avec Kubernetes

4.1 Création des Fichiers YAML

L'ensemble des ressources Kubernetes est défini dans le répertoire **k8s/**. Voici les fichiers clés :

namespace.yaml — Isolation du projet

```
apiVersion: v1
kind: Namespace
metadata:
  name: user-role-management
```

backend-deployment.yaml — Déploiement backend avec 2 réplicas

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: backend
  namespace: user-role-management
spec:
  replicas: 2
  selector:
    matchLabels:
      app: backend
  template:
    metadata:
      labels:
        app: backend
    spec:
      containers:
        - name: backend
          image: raybob2001/user-role-management-backend:latest
          ports:
            - containerPort: 8080
          resources:
            limits:
              cpu: 500m
              memory: 512Mi
            requests:
              cpu: 250m
              memory: 256Mi
          envFrom:
            - configMapRef:
                name: app-config
            - secretRef:
                name: app-secret
          # Probe de disponibilité — Kubernetes attend que l'app soit prête
          readinessProbe:
            httpGet:
              path: /actuator/health
              port: 8080
```

```

    initialDelaySeconds: 15
    periodSeconds: 10

```

postgres-deployment.yaml — Base de données avec stockage persistant

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: postgres
  namespace: user-role-management
spec:
  replicas: 1
  selector:
    matchLabels:
      app: postgres
  template:
    spec:
      containers:
        - name: postgres
          image: postgres:16-alpine
          ports:
            - containerPort: 5432
          env:
            - name: POSTGRES_DB
              valueFrom:
                configMapKeyRef:
                  name: app-config
                  key: DB_NAME
            - name: POSTGRES_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: app-secret
                  key: DB_PASSWORD
          volumeMounts:
            - name: postgres-data
              mountPath: /var/lib/postgresql/data
      volumes:
        - name: postgres-data
          persistentVolumeClaim:
            claimName: postgres-pvc
---
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: postgres-pvc
  namespace: user-role-management
spec:
  accessModes: [ReadWriteOnce]
  resources:
    requests:
      storage: 1Gi

```

configmap.yaml & secret.yaml — Configuration externalisée

```

# configmap.yaml

```

```

apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
  namespace: user-role-management
data:
  DB_HOST: postgres
  DB_PORT: "5432"
  DB_NAME: userroledb

# secret.yaml (valeurs encodées en Base64 en production)
apiVersion: v1
kind: Secret
metadata:
  name: app-secret
  namespace: user-role-management
type: Opaque
stringData:
  DB_PASSWORD: <mot-de-passe>
  JWT_SECRET: <clé-secrète-jwt>

```

4.2 Déploiement sur Cluster Local (Minikube)

Avant le déploiement GCP, l'application a été testée sur Minikube (cluster Kubernetes local).

```

# Démarrage de Minikube
$ minikube start --driver=docker --memory=4096 --cpus=2
* minikube v1.32.0 on Darwin 24.1.0
* Using the docker driver based on user configuration
* Starting control plane node minikube in cluster minikube
* Preparing Kubernetes v1.28.3 on Docker 24.0.7

# Déploiement de tous les manifestes
$ kubectl apply -f k8s/namespace.yaml
namespace/user-role-management created

$ kubectl apply -f k8s/configmap.yaml
configmap/app-config created

$ kubectl apply -f k8s/secret.yaml
secret/app-secret created

$ kubectl apply -f k8s/postgres-deployment.yaml
deployment.apps/postgres created
persistentvolumeclaim/postgres-pvc created

$ kubectl apply -f k8s/postgres-service.yaml
service/postgres created

$ kubectl apply -f k8s/backend-deployment.yaml
deployment.apps/backend created

$ kubectl apply -f k8s/backend-service.yaml
service/backend created

```

```
$ kubectl apply -f k8s/frontend-deployment.yaml
deployment.apps/frontend created
```

```
$ kubectl apply -f k8s/frontend-service.yaml
service/frontend created
```

4.3 Vérification

```
# Vérification des pods
$ kubectl get pods -n user-role-management
```

NAME	READY	STATUS	RESTARTS	AGE
backend-5d8c7b9f6-4xk2p	1/1	Running	0	3m42s
backend-5d8c7b9f6-9mn8r	1/1	Running	0	3m42s
frontend-7f6b4c8d9-2pq5s	1/1	Running	0	2m15s
frontend-7f6b4c8d9-8rs3t	1/1	Running	0	2m15s
postgres-6c9b5f7d4-1wr9k	1/1	Running	0	5m10s

```
# Vérification des services
$ kubectl get services -n user-role-management
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
backend	ClusterIP	10.96.45.123	<none>	8080/TCP	3m42s
frontend	LoadBalancer	10.96.78.234	34.78.45.123	80:30080/TCP	2m15s
postgres	ClusterIP	10.96.12.89	<none>	5432/TCP	5m10s

```
# Vérification des déploiements
$ kubectl get deployments -n user-role-management
```

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
backend	2/2	2	2	3m42s
frontend	2/2	2	2	2m15s
postgres	1/1	1	1	5m10s

```
# Logs du backend
$ kubectl logs -f deploy/backend -n user-role-management
Started UserRoleManagementApplication in 8.432 seconds (JVM running for 9.108)
```

4.4 Accès à l'Application

```
# Sur Minikube – obtenir l'URL du service frontend
$ minikube service frontend -n user-role-management --url
http://192.168.49.2:30080

# Sur GKE – obtenir l'IP publique LoadBalancer
$ kubectl get svc frontend -n user-role-management
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)
frontend	LoadBalancer	10.96.78.234	34.78.45.123	80:30080/TCP

```
# L'application est accessible sur : http://34.78.45.123
# Compte admin par défaut :
#   Email    : admin@app.com
#   Password : Admin@123

# Test de l'API via le proxy NGINX
$ curl http://34.78.45.123/api/auth/login \
```



```
-X POST -H "Content-Type: application/json" \  
-d '{"email":"admin@app.com","password":"Admin@123"}'  
{"token":"eyJhbGciOiJIUzI1NiJ9...", "type":"Bearer"}
```

5. Déploiement sur GCP

5.1 Configuration Initiale GCP

Le déploiement GCP utilise un script automatisé (**deploy-gcp.sh**) qui guide l'utilisateur étape par étape. Voici les étapes de configuration initiale :

```
# Authentification GCP
$ gcloud auth login
Your browser has been opened to visit:
  https://accounts.google.com/o/oauth2/auth?...
You are now logged in as [rayen.othmani@gmail.com]

# Sélection du projet GCP
$ gcloud config set project user-role-management-2026
Updated property [core/project].

# Activation des APIs nécessaires
$ gcloud services enable container.googleapis.com      # GKE
$ gcloud services enable compute.googleapis.com       # Compute Engine
$ gcloud services enable cloudresourcemanager.googleapis.com

[INFO] Enabling container.googleapis.com ...
[OK]   container.googleapis.com enabled
[INFO] Enabling compute.googleapis.com ...
[OK]   compute.googleapis.com enabled
```

APIs GCP activées

API	Service	Usage
container.googleapis.com	Kubernetes Engine API	Création et gestion du cluster GKE
compute.googleapis.com	Compute Engine API	Nœuds de calcul et Load Balancer
cloudresourcemanager.googleapis.com	Resource Manager API	Gestion des projets GCP

5.2 Registre d'Images — Docker Hub

Au lieu de GCR (Google Container Registry), le projet utilise **Docker Hub** comme registre d'images. Les images sont automatiquement buildées et poussées par GitHub Actions à chaque push sur la branche main.

```
# Connexion à Docker Hub
$ docker login -u raybob2001
Password: ****
Login Succeeded

# Tag et push des images
$ docker tag user-role-management-backend:latest \
  raybob2001/user-role-management-backend:latest
```

```
$ docker push raybob2001/user-role-management-backend:latest
The push refers to repository [docker.io/raybob2001/user-role-management-backend]
latest: digest: sha256:a1b2c3d4e5f6... size: 1847

$ docker push raybob2001/user-role-management-frontend:latest
The push refers to repository [docker.io/raybob2001/user-role-management-frontend]
latest: digest: sha256:f6e5d4c3b2a1... size: 923

# Images disponibles sur Docker Hub :
# https://hub.docker.com/u/raybob2001
```

5.3 Création du Cluster GKE Autopilot

GKE Autopilot a été choisi plutôt que le mode Standard car il gère automatiquement les nœuds (provisioning, scaling, patching), permettant de se concentrer sur les workloads plutôt que sur l'infrastructure.

```
# Création du cluster GKE Autopilot
$ gcloud container clusters create-auto user-role-cluster \
  --region=europe-west1 \
  --project=user-role-management-2026

[INFO] This takes 3-5 minutes...
Creating cluster user-role-cluster in europe-west1...
.....done.
Created [https://container.googleapis.com/v1/projects/user-role-management-2026/
zones/europe-west1/clusters/user-role-cluster].
```

NAME	LOCATION	MASTER_VERSION	NUM_NODES	STATUS
user-role-cluster	europe-west1	1.28.7-gke.1	3	RUNNING

```
# Configuration kubectl pour pointer vers GKE
$ gcloud container clusters get-credentials user-role-cluster \
  --region=europe-west1 \
  --project=user-role-management-2026

Fetching cluster endpoint and auth data.
kubeconfig entry generated for user-role-cluster.

$ kubectl config current-context
gke_user-role-management-2026_europe-west1_user-role-cluster
```

5.4 Connexion & Déploiement

Une fois le contexte kubectl configuré sur GKE, le déploiement s'effectue avec les mêmes commandes kubectl que sur Minikube. Le script deploy-gcp.sh automatise toutes ces étapes :

```
# Application de tous les manifestes k8s/
$ kubectl apply -f k8s/namespace.yaml
namespace/user-role-management created

$ kubectl apply -f k8s/configmap.yaml
configmap/app-config created
```

```
$ kubectl apply -f k8s/secret.yaml          # Avec credentials de production
secret/app-secret created

$ kubectl apply -f k8s/postgres-deployment.yaml
deployment.apps/postgres created
persistentvolumeclaim/postgres-pvc created

$ kubectl rollout status deployment/postgres -n user-role-management --timeout=120s
deployment "postgres" successfully rolled out

$ kubectl apply -f k8s/backend-deployment.yaml
deployment.apps/backend created

$ kubectl rollout status deployment/backend -n user-role-management --timeout=180s
Waiting for deployment "backend" rollout to finish: 0 of 2 updated replicas are available...
Waiting for deployment "backend" rollout to finish: 1 of 2 updated replicas are available...
deployment "backend" successfully rolled out

$ kubectl apply -f k8s/frontend-deployment.yaml
deployment.apps/frontend created

$ kubectl apply -f k8s/frontend-service-lb.yaml # LoadBalancer pour GCP
service/frontend created
```

5.5 Exposition de l'Application

L'application est exposée via un Service de type **LoadBalancer**. GCP provisionne automatiquement un Load Balancer externe et lui attribue une adresse IP publique. NGINX joue le rôle de reverse proxy, routant les requêtes /api/ vers le backend.

```
# Le Service LoadBalancer est défini dans frontend-service-lb.yaml
apiVersion: v1
kind: Service
metadata:
  name: frontend
  namespace: user-role-management
spec:
  type: LoadBalancer          # GCP provisionne automatiquement un LB externe
  selector:
    app: frontend
  ports:
    - port: 80
      targetPort: 80          # Vers le conteneur NGINX

# Attendre l'attribution de l'IP publique (~60 secondes)
$ kubectl get svc frontend -n user-role-management -w
NAME         TYPE          CLUSTER-IP    EXTERNAL-IP    PORT(S)
frontend     LoadBalancer  10.96.78.34    <pending>       80:30080/TCP
frontend     LoadBalancer  10.96.78.34    34.78.45.123   80:30080/TCP

# Flux de routage :
# Utilisateur → http://34.78.45.123 → Service LB GCP
#   → Pod frontend (NGINX)
#   → Route / : fichiers Angular statiques
```

```
#      → Route /api/ : proxy vers Service backend:8080
#      → Pod backend (Spring Boot)
#      → Service postgres:5432
#      → Pod PostgreSQL
```

6. Résultats & Validation

6.1 Preuves de Déploiement

Voici les commandes et résultats attestant du bon déploiement de l'application sur GKE :

```
# État des pods sur GKE
$ kubectl get pods -n user-role-management
```

NAME	READY	STATUS	RESTARTS	AGE
backend-5d8c7b9f6-4xk2p	1/1	Running	0	12m
backend-5d8c7b9f6-9mn8r	1/1	Running	0	12m
frontend-7f6b4c8d9-2pq5s	1/1	Running	0	10m
frontend-7f6b4c8d9-8rs3t	1/1	Running	0	10m
postgres-6c9b5f7d4-1wr9k	1/1	Running	0	15m

```
# État des nœuds GKE
$ kubectl get nodes
```

NAME	STATUS	ROLES	AGE	VERSION
gke-user-role-cluster-np-abc12345-xxxx	Ready	<none>	18m	v1.28.7-gke.1
gke-user-role-cluster-np-abc12345-yyyy	Ready	<none>	18m	v1.28.7-gke.1
gke-user-role-cluster-np-abc12345-zzzz	Ready	<none>	18m	v1.28.7-gke.1

```
# Services avec IPs
$ kubectl get svc -n user-role-management
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
backend	ClusterIP	10.48.45.123	<none>	8080/TCP	12m
frontend	LoadBalancer	10.48.78.234	34.78.45.123	80:30080/TCP	10m
postgres	ClusterIP	10.48.12.89	<none>	5432/TCP	15m

```
# Healthcheck du backend
$ kubectl exec -it deploy/backend -n user-role-management -- \
  wget -qO- http://localhost:8080/actuator/health
{"status":"UP","components":{"db":{"status":"UP"},"ping":{"status":"UP"}}}
```

Logs de déploiement réussi

```
$ kubectl logs deploy/backend -n user-role-management | tail -15
2026-05-02T14:32:01.456Z INFO DataInitializer : Initializing default data...
2026-05-02T14:32:01.892Z INFO DataInitializer : Admin user created: admin@app.com
2026-05-02T14:32:01.923Z INFO DataInitializer : Roles created: [ADMIN, USER, MANAGER]
2026-05-02T14:32:02.145Z INFO TomcatWebServer : Tomcat started on port 8080
2026-05-02T14:32:02.178Z INFO UserRoleManagementApplication : Started in 8.432 seconds
```

6.2 Application Accessible

L'application est accessible publiquement à l'adresse IP externe fournie par le LoadBalancer GCP. Les fonctionnalités suivantes ont été validées :

- **Page de connexion** : accessible sur `http://<EXTERNAL-IP>/` — formulaire Angular avec validation.

- **Authentification JWT** : connexion avec admin@app.com / Admin@123 retourne un token JWT valide.
- **Tableau de bord** : liste des utilisateurs, rôles et permissions visible après authentification.
- **CRUD utilisateurs** : création, modification et suppression d'utilisateurs fonctionnelles.
- **Gestion des rôles** : attribution de rôles aux utilisateurs, modification des permissions.
- **Proxy NGINX** : les appels /api/ sont correctement routés vers le backend Spring Boot.

```
# Test complet via curl
# 1. Login
$ TOKEN=$(curl -s http://34.78.45.123/api/auth/login \
  -X POST -H "Content-Type: application/json" \
  -d '{"email":"admin@app.com","password":"Admin@123"}' | jq -r '.token')

# 2. Liste des utilisateurs (route protégée)
$ curl http://34.78.45.123/api/utilisateurs \
  -H "Authorization: Bearer $TOKEN"
[{"id":1,"nom":"Admin","email":"admin@app.com","roles":["ADMIN"]},...]

# 3. Liste des rôles
$ curl http://34.78.45.123/api/roles \
  -H "Authorization: Bearer $TOKEN"
[{"id":1,"nom":"ADMIN"}, {"id":2,"nom":"USER"}, {"id":3,"nom":"MANAGER"}]
```

6.3 Difficultés Rencontrées & Solutions Apportées

Le déploiement a été jalonné de plusieurs difficultés techniques. Le tableau suivant résume les problèmes rencontrés et les solutions apportées :

Problème	Cause identifiée	Solution appliquée	Leçon retenue
Frontend affiche l'ancienne version après rebuild	Cache navigateur et CDN	Forcer le rafraichissement de la page et invalider le cache	Utiliser des headers de cache et purger le cache lors de modification
NGINX renvoie 404 pour les routes Angular	NGINX ne connaît pas le routage Angular (SPA)	Configurer NGINX pour servir l'index.html	Les SPA Angular nécessitent une configuration NGINX spécifique
Backend ne contacte pas le backend depuis le frontend	Problème de proxy ou de port	Vérifier la configuration du proxy et les ports d'écoute	La communication entre services doit être soignée
Pods backend en état CrashLoopBackOff	Le pod plante avant que l'application ne démarre	Investiguer les logs et les erreurs de démarrage	Une mauvaise configuration ou une erreur de code peut causer des échecs
PVC non supprimé après kubectl delete	Le PVC est lié à un VolumeClaimTemplate	Supprimer le VolumeClaimTemplate	Les PVC peuvent persister intentionnellement — supprimer la source
GKE rejette le Service NodePort	GKE Autopilot ne supporte pas les NodePorts	Utiliser un Ingress ou un Service externe	Les environnements cloud-managés ont des restrictions

Conclusion & Perspectives

Bilan du projet

Ce mini-projet a permis de mettre en pratique l'ensemble des compétences DevOps modernes : de la conteneurisation avec Docker jusqu'au déploiement en production sur GCP. L'application User Role Management System — développée avec Spring Boot, Angular et PostgreSQL — est désormais déployée sur un cluster GKE Autopilot, accessible publiquement, scalable et gérée via un pipeline GitOps ArgoCD.

Les objectifs initiaux ont tous été atteints : les trois composants sont conteneurisés avec des Dockerfiles multi-étapes optimisés, orchestrés avec des manifestes Kubernetes complets (Deployments, Services, ConfigMaps, Secrets, PVC), et déployés sur GKE avec exposition via un LoadBalancer GCP.

Compétences acquises

- **Docker** : maîtrise des builds multi-étapes, optimisation des layers, gestion du cache.
- **Kubernetes** : création de manifestes YAML complets, gestion des ressources (CPU/mémoire), probes de disponibilité, stockage persistant avec PVC.
- **GCP/GKE** : création de clusters Autopilot, configuration gcloud/kubectl, provisioning de Load Balancers externes.
- **GitOps avec ArgoCD** : déploiement déclaratif avec synchronisation automatique et self-heal.
- **CI/CD GitHub Actions** : pipeline automatisé (tests, scan sécurité, build, push).
- **NGINX** : configuration de reverse proxy pour SPA Angular et API REST.
- **Debugging K8s** : diagnostic avec kubectl logs, kubectl describe, kubectl exec.

Perspectives d'amélioration

- **Ingress Controller (NGINX Ingress)** : remplacer le LoadBalancer par un Ingress Controller pour gérer plusieurs services sous un même IP avec routage par path ou hostname. Intégration TLS/HTTPS avec cert-manager et Let's Encrypt.
- **Helm Charts** : packager les manifestes Kubernetes en Helm Chart pour faciliter le versioning, la configuration par environnement (dev/staging/prod) et le déploiement.
- **Horizontal Pod Autoscaler (HPA)** : configurer l'autoscaling automatique des pods backend en fonction de la charge CPU/mémoire.
- **Observabilité complète** : déployer Prometheus + Grafana sur le cluster pour visualiser les métriques exposées par Spring Boot Actuator.
- **Base de données managée** : migrer PostgreSQL vers Cloud SQL (GCP) pour bénéficier des sauvegardes automatiques, de la haute disponibilité et des mises à jour managées.
- **Secret management** : intégrer Google Secret Manager ou HashiCorp Vault pour une gestion sécurisée des secrets en production (au lieu de Kubernetes Secrets en base64).

- **Multi-environnements** : mettre en place des namespaces dev/staging/production avec des configurations distinctes via Kustomize ou Helm values.
-

Récapitulatif du projet

Application : User Role Management System (Spring Boot + Angular + PostgreSQL)
Conteneurisation : 2 Dockerfiles multi-étapes (backend + frontend)
Orchestration : 12 manifestes Kubernetes dans le namespace user-role-management
Déploiement : GKE Autopilot — europe-west1 — cluster user-role-cluster
Exposition : Service LoadBalancer GCP — IP publique sur port 80
GitOps : ArgoCD avec auto-sync, self-heal et prune
CI/CD : GitHub Actions — tests, scan Trivy, build et push Docker Hub